

MALWARE BEHAVIOR ANALYSIS

Command & Control Simulation and Post-Exploitation Investigation

Malware Analysis / Blue Team Project Documentation

Author

Shivam

Date

August 2026

Technologies Used

Kali Linux | Windows 7 (Victim VM) | VMware Workstation
PowerShell Empire 6.0.2 | Starkiller 3.0.0 | Python HTTP Server
Sysinternals Suite | Command & Control (C2) Framework

Category: Malware Analysis | Post-Exploitation | Blue Team | Threat Investigation

2. Project Overview

2.1 What is Malware Behavior Analysis

Malware Behavior Analysis is the practice of executing or simulating malicious software in a controlled environment to observe how it delivers itself, communicates with an attacker, and interacts with the host system. Rather than relying only on static signatures, behavioral analysis focuses on real-world actions — network callbacks, file system access, and command execution — which is the same evidence a SOC analyst reviews during an incident.

2.2 Purpose

This project simulates a complete attack chain inside an isolated virtual lab: staging payload samples, standing up a Command & Control (C2) listener, delivering a payload to a Windows victim, and investigating the resulting agent session, exactly as an attacker (and subsequently an analyst) would.

2.3 Objectives

- Understand the behavioral characteristics of common malware categories.
- Deploy a Command & Control (C2) infrastructure using PowerShell Empire and Starkiller.
- Host and deliver a payload to a victim machine using a Python HTTP server.
- Observe and validate an agent callback (beaconing) from the compromised host.
- Interact with a live agent session and issue post-exploitation commands.
- Investigate the compromised file system remotely to identify attacker visibility and evidence.
- Document findings from both a malware analyst and SOC analyst perspective.

3. Malware Fundamentals

The lab exercise references several core malware categories to contextualize how the simulated payload behaves compared to real-world threats:

- Virus — attaches to legitimate files/programs and requires user action to activate; corrupts or deletes data.
- Worm — self-replicates and spreads across networks without user interaction, consuming bandwidth and opening backdoors.
- Trojan — disguised as legitimate software; the user unknowingly installs it, allowing data theft or a backdoor to be opened. The payload used in this lab behaves as a Trojan-style dropper/stager.
- Ransomware — delivered via phishing or drive-by downloads; encrypts files and demands payment for decryption.
- Rootkit — bundled with software or installed via Trojans/exploits to hide malware presence and grant admin-level control.

Malware Type	Spreads By	Harm Type	Famous Case
Virus	Attaches to legitimate files or programs; needs user action to activate	Corrupts or deletes files, slows down system	Melissa Virus (1999) – Spread via Word docs and Outlook
Worm	Self-replicates and spreads via network or email without user action	Consumes bandwidth, overloads systems, opens backdoors	ILOVEYOU (2000), Conficker (2008)
Trojan	Disguised as legitimate software; user unknowingly installs it	Steals data, opens backdoor, downloads other malware	Zeus Trojan (2007) – Stole banking credentials
Ransomware	Phishing emails, malicious links, drive-by downloads	Encrypts files, demands ransom in cryptocurrency	WannaCry (2017), Colonial Pipeline (2021)
Rootkit	Bundled with software, installed via Trojans, exploits	Hides presence of malware, gives admin-level control	Sony BMG CD Rootkit Scandal (2005)

Figure 1: Malware Category Reference — Spread Method, Impact, and Notable Cases

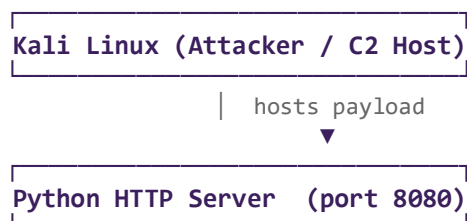
This reference table summarizes how each malware category spreads, the harm it causes, and a real-world example (e.g., WannaCry for ransomware, Zeus for Trojans). Understanding these distinctions helps a SOC analyst quickly classify a threat during triage and anticipate its likely next behavior.

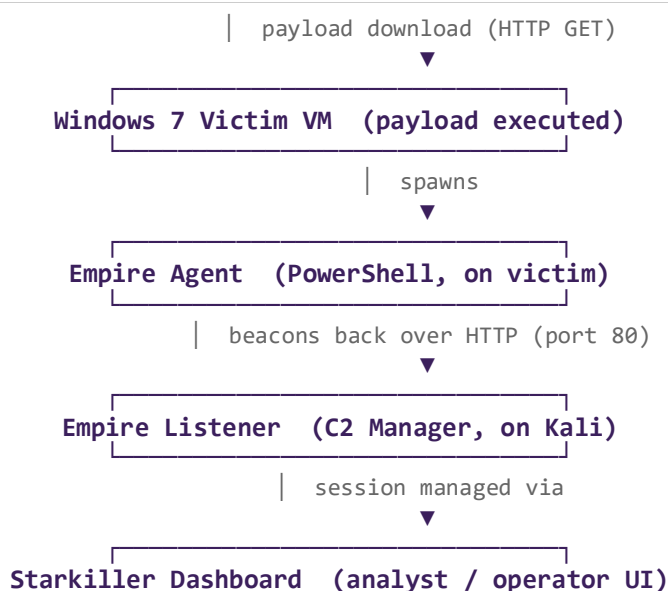
4. Lab Environment

Component	Details
Host Machine	Physical workstation running VMware Workstation Pro
Attacker Machine	Kali Linux 2025.1a (VMware VM)
Victim Machine	Windows 7 x64 (VMware VM)
Virtualization	VMware Workstation
Empire Framework	PowerShell Empire 6.0.2 — Command & Control (C2) backend
Starkiller	Starkiller 3.0.0 — Web-based GUI client for Empire
Python HTTP Server	Python 3.13 built-in http.server module — payload hosting on port 8080
Operating Systems	Kali Linux (attacker), Windows 7 x64 (victim), Metasploitable2-Linux (auxiliary target)
Network Type	Isolated host-only / NAT virtual network (192.168.26.0/24, 192.168.88.0/24)

5. Lab Architecture

The lab implements a standard Command & Control (C2) topology consisting of an attacker-controlled framework, a payload delivery channel, and a victim endpoint:





5.1 Communication Flow

- The Kali attacker machine hosts a malicious stager on a lightweight Python HTTP server.
- The Windows 7 victim retrieves the payload over HTTP and executes it locally.
- Execution spawns a PowerShell-based Empire agent process on the victim.
- The agent initiates outbound beaconing to the Empire Listener on port 80, establishing the C2 channel.
- The Empire Manager registers the session and exposes it through the Starkiller web dashboard, allowing the operator to interact with the compromised host.

6. Project Workflow

Step 1 — Preparing Malware Samples

Purpose: Stage a set of executable samples and legitimate dual-use tools to be used for payload testing and post-exploitation activity.

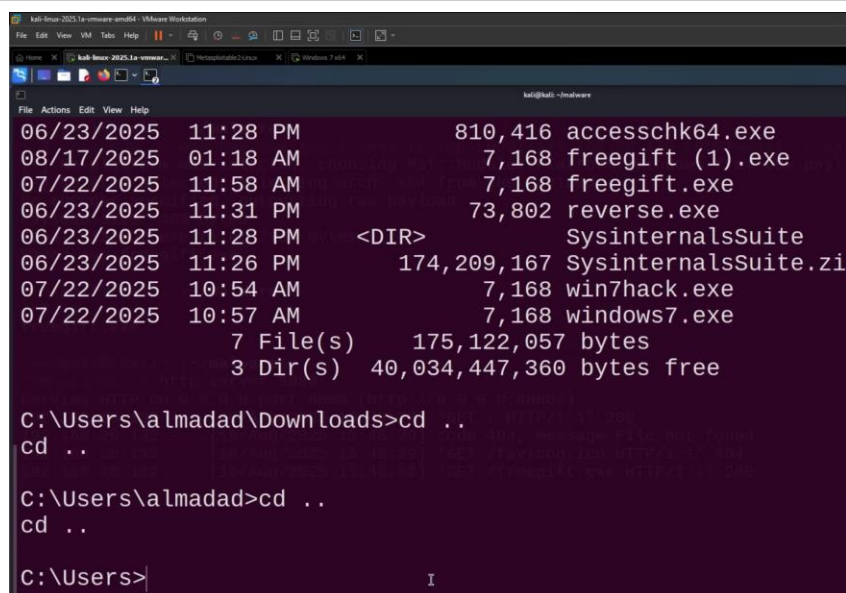
Implementation: A working directory (~/.malware) was created on the Kali attacker machine containing staged binaries, including a custom stager (freegift.exe), a reverse-shell payload (reverse.exe), renamed test payloads (win7hack.exe, windows7.exe), and the Sysinternals Suite for later use as a legitimate administrative toolset.

Expected Result: All required sample files are present and accessible for hosting and delivery.

Observed Result: Directory listing confirms seven files/folders totalling ~175 MB are staged and ready, matching the naming convention used in later delivery steps.

Associated Screenshot: Figure 2

SOC Analyst Perspective: Analysts should note that filenames alone (e.g., "freegift.exe") are a common social-engineering lure; file size, hash, and creation timestamp — not the name — are the reliable indicators used for triage.



```

kali@kali:~/Downloads$ ls -l
total 175
-rwxr-xr-x 1 kali kali 810,416 06/23/2025 11:28 PM accesschk64.exe
-rwxr-xr-x 1 kali kali 7,168 08/17/2025 01:18 AM freegift (1).exe
-rwxr-xr-x 1 kali kali 7,168 07/22/2025 11:58 AM freegift.exe
-rwxr-xr-x 1 kali kali 73,802 06/23/2025 11:31 PM reverse.exe
-rwxr-xr-x 1 kali kali 174,209,167 06/23/2025 11:26 PM SysinternalsSuite.zip
-rwxr-xr-x 1 kali kali 7,168 07/22/2025 10:54 AM win7hack.exe
-rwxr-xr-x 1 kali kali 7,168 07/22/2025 10:57 AM windows7.exe
7 File(s) 175,122,057 bytes
3 Dir(s) 40,034,447,360 bytes free

C:\Users\almadad\Downloads>cd ..
cd ..

C:\Users\almadad>cd ..
cd ..

C:\Users>

```

Figure 2: Staged Malware Sample Directory on Attacker Machine

This terminal output lists the payload and tool files staged in the attacker's working directory, including the stager (freegift.exe), reverse-shell binary (reverse.exe), and the Sysinternals Suite. Reviewing staged tooling like this is a key step during incident response when reconstructing an attacker's toolkit from a compromised host or C2 server.

Step 2 — Creating the Empire Listener

Purpose: Establish the Command & Control channel that will receive callbacks from the payload once executed on the victim.

Implementation: An HTTP listener was created in Empire via the Starkiller "Listeners" panel, bound to the Kali host IP (192.168.26.130) on port 80.

Expected Result: The listener starts successfully and shows an active (green) status, ready to accept incoming agent connections.

Observed Result: The listener "http" is shown active, using the http template, bound to http://192.168.26.130:80, created and running without errors.

Associated Screenshot: Figure 3

SOC Analyst Perspective: Listener configuration (host, port, template) observed here is exactly the kind of infrastructure fingerprint analysts hunt for in outbound proxy/firewall logs and threat-intel feeds when identifying C2 traffic.

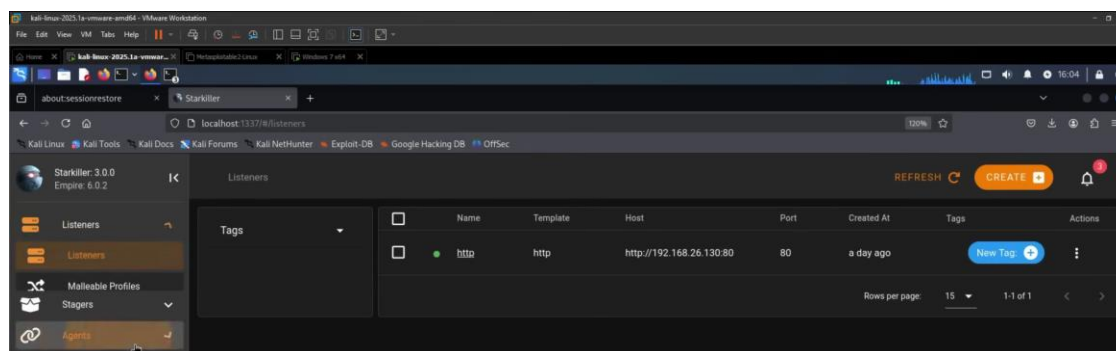


Figure 3: Active Empire Listener Configured in Starkiller

The Starkiller Listeners panel confirms an HTTP listener is active on the attacker machine (192.168.26.130:80). This is the C2 endpoint the payload will beacon back to once executed — validating listener status is a prerequisite before payload delivery, since a failed listener would prevent any agent callback.

Step 3 — Hosting Payload using Python HTTP Server

Purpose: Provide a delivery mechanism for the victim machine to retrieve the Empire stager over HTTP.

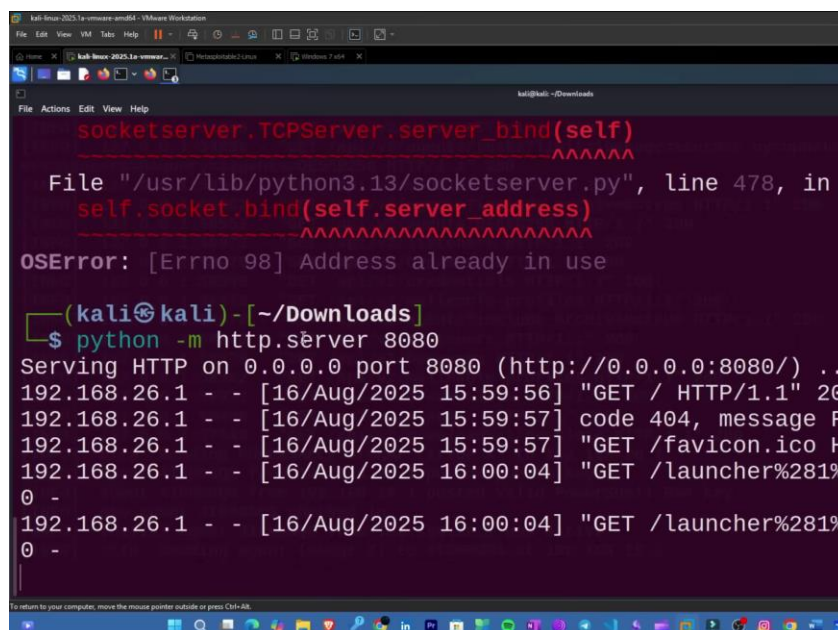
Implementation: Python's built-in web server module was started in the payload directory using `python -m http.server 8080`. An initial attempt failed with `OSError: [Errno 98] Address already in use`, indicating a prior server instance was still bound to the port; the server was restarted successfully afterward.

Expected Result: The server binds to port 8080 and serves incoming GET requests for the stager file.

Observed Result: Access logs show successful requests from 192.168.26.1, including GET /launcher... calls consistent with an Empire PowerShell launcher being downloaded and executed by the victim.

Associated Screenshot: Figure 4

SOC Analyst Perspective: Unusual outbound HTTP GET requests for files named "launcher" on non-standard ports, followed shortly by new process creation, are classic indicators of payload staging worth alerting on in proxy/web logs.



```

kali@kali:~/Downloads$ python -m http.server 8080
OSError: [Errno 98] Address already in use

(kali@kali) - [~/Downloads]
$ python -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.26.1 - - [16/Aug/2025 15:59:56] "GET / HTTP/1.1" 200
192.168.26.1 - - [16/Aug/2025 15:59:57] code 404, message File not found
192.168.26.1 - - [16/Aug/2025 15:59:57] "GET /favicon.ico HTTP/1.1" 404
192.168.26.1 - - [16/Aug/2025 16:00:04] "GET /launcher%281% HTTP/1.1" 200
0 -
192.168.26.1 - - [16/Aug/2025 16:00:04] "GET /launcher%281% HTTP/1.1" 200
0 -

```

Figure 4: Python HTTP Server Hosting the Payload (Port 8080)

The terminal shows the Python HTTP server successfully serving the payload after resolving a port-conflict error, with access logs recording the victim machine's GET requests for the launcher stager. This log is direct evidence of payload delivery and is the type of artifact reviewed when tracing an intrusion back to its initial access vector.

Step 4 — Executing Payload

Purpose: Trigger execution of the delivered stager on the Windows 7 victim to establish the initial foothold.

Implementation: The downloaded launcher stager was executed on the Windows 7 victim VM under the user context almadad, invoking PowerShell to run the Empire stager code in memory.

Expected Result: Execution completes silently and initiates an outbound connection attempt to the configured listener.

Observed Result: The HTTP server access log (Figure 4) confirms two sequential GET /launcher requests from the victim IP, consistent with the stager being downloaded and executed.

Associated Screenshot: Figure 4 (server-side evidence of execution/delivery)

SOC Analyst Perspective: Because the stager runs in memory via PowerShell, it is unlikely to be captured by traditional file-based antivirus; analysts should rely on PowerShell script-block logging and process-creation telemetry to detect this stage.

Step 5 — Agent Callback

Purpose: Confirm that the executed payload successfully beacons back and registers a live session with the Empire Manager.

Implementation: After execution, the Starkiller Agents panel was monitored for new session registrations.

Expected Result: A new agent entry appears, showing hostname, process, language, and internal IP of the compromised host.

Observed Result: Two active PowerShell agents (OE55B258 and 4I0M6GQS) are shown from host DESKTOP-TUU6T1U, user almadad, internal IP 192.168.88.1 — confirming successful callback.

Associated Screenshot: Figure 5

SOC Analyst Perspective: The "Last Seen" / "First Seen" beaconing interval and consistent hostname/user pairing are key data points for building a timeline of compromise and confirming persistence across sessions.

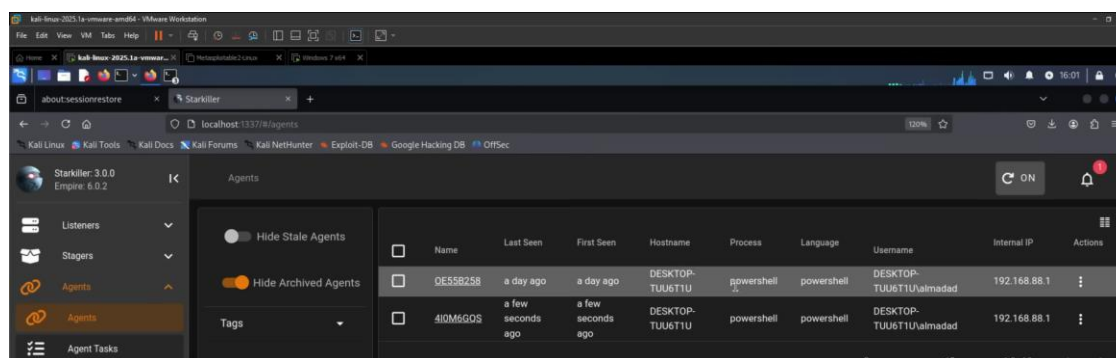


Figure 5: Empire Agents Registered After Successful Callback

The Starkiller Agents dashboard lists two live PowerShell agents originating from the same victim host (DESKTOP-TUU6T1U), confirming the payload executed successfully and established a C2 channel back to the attacker. Agent metadata such as hostname, username, and internal IP is critical for scoping the blast radius of a compromise.

Step 6 — Interacting with the Agent

Purpose: Demonstrate operator control over the compromised host through the C2 session.

Implementation: The Interact tab was opened for agent 4I0M6GQS, exposing both a shell-command form and a module-execution form for issuing post-exploitation commands.

Expected Result: Commands submitted through the interface are relayed to the agent and executed on the victim, with results returned to the operator.

Observed Result: The Interact panel confirms a live, addressable session (agent ID 4I0M6GQS) ready to accept shell commands or Empire modules.

Associated Screenshot: Figure 6

SOC Analyst Perspective: Every command issued through this interface generates a corresponding task/result pair on the agent, which — if the C2 traffic or endpoint logs are captured — forms a reconstructable timeline of attacker actions during incident response.

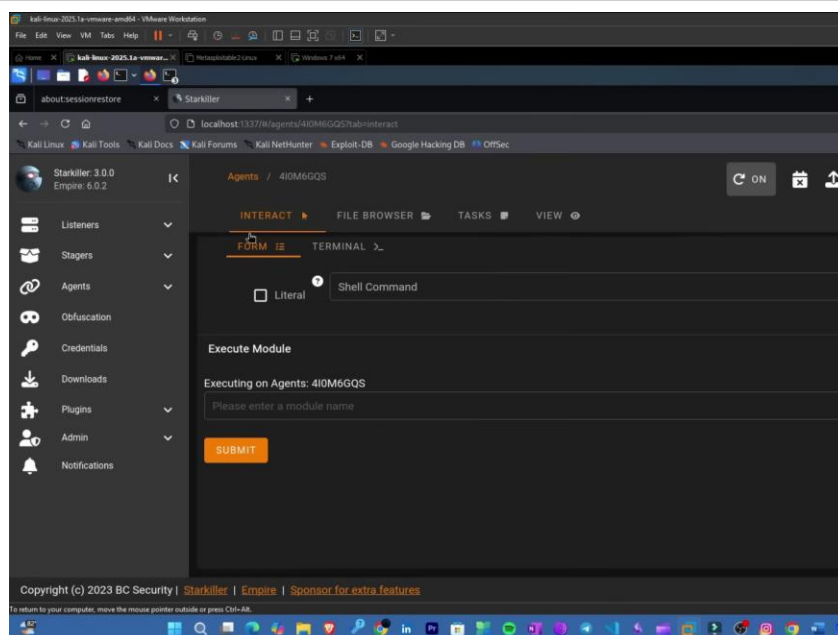


Figure 6: Starkiller Interact Panel for Live Agent Session

This view shows the Interact tab for agent 4IOM6GQS, providing both a shell command field and a module execution form used to control the compromised endpoint. This is the operator's primary post-exploitation control surface, equivalent to what an analyst must assume an attacker has once an agent is confirmed active.

Step 7 — Remote File System Exploration

Purpose: Assess what an attacker can see and access on the compromised host once a session is established.

Implementation: The File Browser tab was used to enumerate the victim's drives, and directory contents were retrieved for further inspection; the corresponding output was cross-checked against the actual victim machine's File Explorer view.

Expected Result: The C2 session should be able to remotely list all mounted drives and browse folder contents without triggering local user interaction.

Observed Result: The remote file browser successfully enumerated drives C:\, D:\, and G:\ and began retrieving the contents of C:\ — matching exactly the three drives (Local Disk C:, Personal D:, Games & Software G:) visible in the victim's native File Explorer.

Associated Screenshot: Figures 7 and 8

SOC Analyst Perspective: Remote drive/directory enumeration immediately following a new agent callback is a strong indicator of post-exploitation reconnaissance; correlating this with file-access audit logs (e.g., Windows Event ID 4663) helps identify what data may have been staged for exfiltration.

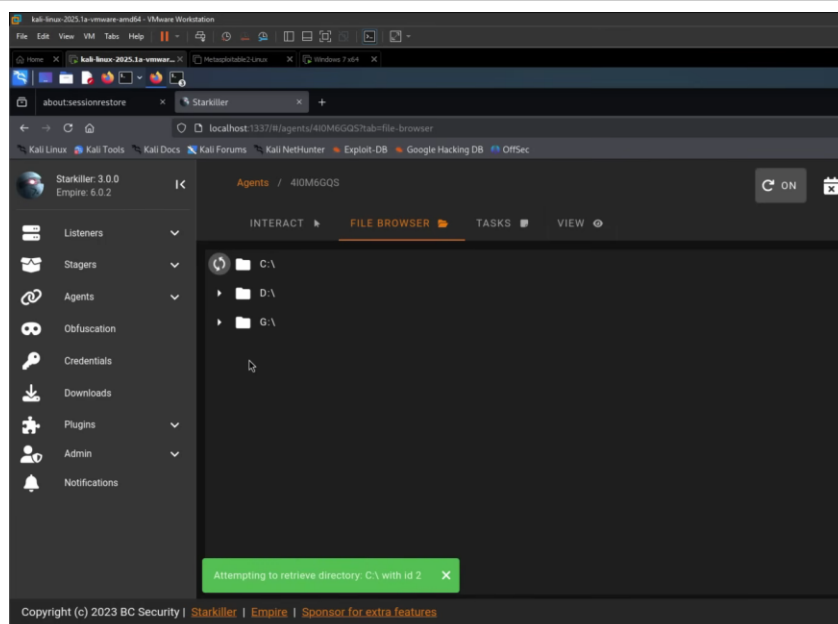


Figure 7: Remote File Browser — Victim Drive Enumeration via Starkiller

The Starkiller File Browser remotely enumerates the victim's C:\, D:\, and G:\ drives and requests the contents of C:\, entirely through the C2 channel with no local interaction required. This capability represents the exact attacker behavior a SOC analyst must detect through endpoint or file-access auditing.

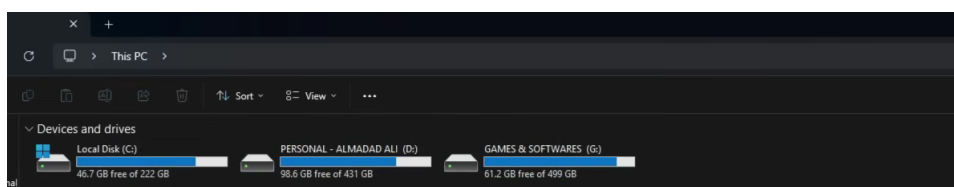


Figure 8: Victim Machine — Native File Explorer Drive View (Ground Truth)

Captured directly on the Windows 7 victim, this view confirms the same three drives — Local Disk (C:), Personal - ALMADAD ALI (D:), and Games & Softwares (G:) — reported by the remote file browser in Figure 7. This side-by-side validation confirms the C2 session's visibility is accurate and unrestricted, with no discrepancy between attacker-observed and ground-truth data.

7. Technical Concepts

Concept	Explanation
Empire Framework	An open-source post-exploitation and adversary-emulation framework that generates PowerShell/Python agents and manages C2 listeners, tasking, and modules.
Starkiller	A web-based GUI client for Empire that simplifies listener creation, payload generation, agent interaction, and file browsing without using the raw command line.
Command & Control (C2)	The infrastructure and protocol an attacker uses to maintain remote communication with, and control over, a compromised host after initial access.
Payload Delivery	The mechanism (in this case, an HTTP-hosted stager) used to transfer malicious code to the target so it can be executed.
Agent Callback	The outbound beacon initiated by executed malware/stager code back to the C2 listener, registering a controllable session.

Concept	Explanation
Post Exploitation	Actions taken after initial compromise — command execution, file access, credential harvesting, persistence — to expand and maintain control.
Remote File Browser	A C2 module/feature that allows an operator to enumerate and browse the victim's file system without direct local access.

These concepts matter during investigations because each maps directly to a detection opportunity: listener infrastructure appears in network telemetry, payload delivery appears in web/proxy logs, agent callbacks appear as periodic beaconing, and post-exploitation/file-browsing actions appear in endpoint audit logs — together forming the evidentiary chain an analyst reconstructs during incident response.

8. Commands Used

Command	Purpose	Description
<code>python -m http.server 8080</code>	Payload hosting	Starts a lightweight HTTP server on port 8080 to serve the Empire stager for download by the victim.
<code>cd ..</code>	Directory navigation	Moves up one directory level; used while navigating the victim's file system through an interactive shell.
<code>dir</code>	Directory listing	Windows command used (via agent shell / native terminal) to list files, sizes, and timestamps in the current directory.
Empire Listener creation (Starkiller UI)	C2 setup	Configures and starts an HTTP listener bound to the attacker IP and port to receive agent callbacks.
Agent Interact — Shell Command (Starkiller UI)	Post-exploitation	Submits an arbitrary shell command to a live agent session for remote execution on the victim.
Agent Interact — Execute Module (Starkiller UI)	Post-exploitation	Runs a named Empire module against the selected agent for extended post-exploitation functionality.
File Browser — directory retrieval (Starkiller UI)	Reconnaissance	Remotely requests and lists the contents of a specified drive/directory (e.g., C:\) on the compromised host.

9. Analysis & Findings

9.1 How the Malware Communicates

The compromised host communicates with the attacker over standard HTTP (port 80), using Empire's built-in listener protocol. This blends with normal web traffic, which is why analysts must inspect request patterns (regular intervals, unusual user agents, consistent destination) rather than relying on port number alone.

9.2 How Payload Delivery Works

Delivery relied on a simple, low-footprint HTTP file server rather than a dedicated exploit — the victim voluntarily retrieved and executed the launcher. This mirrors real-world drive-by-download and social-engineering delivery, where the exploit is the download-and-run action itself, not a software vulnerability.

9.3 How the Agent Establishes Its Session

Once executed, the stager runs entirely in memory via PowerShell and immediately beacons to the configured listener, registering itself as a new agent with the manager. No persistence mechanism (e.g., scheduled task, registry run key) was configured in this lab, meaning the session would not survive a reboot — a detail worth confirming during any real triage.

9.4 How Remote Interaction Was Achieved

The Starkiller Interact panel relays operator-submitted commands to the agent over the same C2 channel used for beaconing, with results returned asynchronously and displayed in the UI — demonstrating full interactive control without a direct remote-desktop or VPN connection.

9.5 Evidence Collected

- Listener configuration (host, port, protocol) from the Empire Manager.
- HTTP access logs from the Python server confirming payload download requests.
- Agent registration metadata (hostname, username, internal IP, process, language).
- Remote file system enumeration output cross-validated against the victim's local view.

9.6 File System Exploration Outcome

Remote enumeration successfully identified all three mounted drives (C:, D:, G:) on the victim and began retrieving folder contents — confirming an attacker with an active agent has full visibility into the victim's local and personal storage without needing any additional privilege escalation.

9.7 What an Analyst Should Investigate at This Stage

- Outbound connections to unfamiliar IPs/domains occurring at regular beacon-like intervals.
- PowerShell process creation with encoded or obfuscated command-line arguments.
- New, unrecognized processes spawned shortly after a browser or email-client download event.
- File-access audit events on sensitive drives/folders immediately following a suspicious process launch.
- Absence or presence of persistence artifacts (registry run keys, scheduled tasks, services).

10. Skills Demonstrated

Category	Skills
Malware Analysis	Malware Behavior Analysis, Malware Investigation, Threat Investigation
C2 Tooling	Empire Framework, Starkiller, Command & Control (C2) Operations
Delivery & Execution	Python HTTP Server, Payload Delivery, Virtual Lab Deployment
Post-Exploitation	Post Exploitation, Windows File System Analysis
Documentation	Cybersecurity Documentation, Technical Reporting

11. Key Learning Outcomes

11.1 From a Malware Analyst Perspective

- Gained hands-on understanding of how a stager-based payload is delivered, executed, and establishes an in-memory C2 session.
- Learned to distinguish behavioral indicators (network beaconing, process spawning) from static file-based indicators.
- Practiced correlating server-side delivery logs with client-side execution and callback evidence.

11.2 From a SOC Analyst Perspective

- Reinforced the value of cross-validating C2/attacker-observed data (remote file browser) against ground-truth endpoint data.
- Identified concrete detection opportunities at every stage of the attack chain — delivery, execution, callback, and post-exploitation.
- Practiced building an evidence-based investigation narrative from fragmented technical artifacts, consistent with real incident response workflows.

12. Conclusion

This project provided practical, end-to-end exposure to a simulated attack lifecycle — from staging and hosting a payload, through Command & Control listener setup, agent callback, and remote post-exploitation file system access — using PowerShell Empire and Starkiller in an isolated lab. By operating both the attacker tooling and analyzing the resulting evidence, the exercise strengthened core competencies in malware behavior analysis, C2 communication patterns, payload delivery mechanics, and post-exploitation investigation, all of which translate directly to real-world SOC detection and incident response work.